

CloudGuardian - Azure Week 3 Setup Guide

Deploying the Remediation, Approval, Governance and Privacy-Preserving LLM Stack

A complete walkthrough, written the same way as the Week 1 setup guide - file by file, then step by step. Assumes Week 1's Terraform stack is already deployed and running.

Part 1 - A Quick Tour of the Project Files

Week 3 adds five new folders on top of Week 1's terraform/ folder: terraform/ (new files only), logic_app/, automation/, functions/, and privacy_llm/. Same rule as Week 1: Terraform reads every .tf file in a folder together as one blueprint - the order below is the order that makes sense to READ them in, not an execution order.

1.1 terraform/ - new files added for Week 3

versions.tf / providers.tf - *version pins, same pattern as Week 1*

Pins azurearm (~> 3.110) and random (~> 3.6). An earlier design also needed the azapi provider to deploy a hand-authored Logic App JSON workflow; that's gone now that the Logic App is a thin native-HCL forwarder (see 1.5 below), so the provider list stayed as simple as Week 1's.

variables.tf - *the settings panel - now points AT Week 1 instead of creating new infrastructure*

Unlike Week 1, almost every variable here is 'existing_...' - the name of a resource Week 1 already built (Resource Group, Log Analytics Workspace, Key Vault, Storage Account, SQL Server/Database). Week 3 does not create a new environment; it attaches automation to the one you already have.

locals.tf - *naming + data-source lookups*

Generates names for Week 3's own new resources (Function App, Event Grid Topic, Logic App, Automation Account, ACS Communication/Email Service) and looks up Week 1's existing resources via `data` blocks so the rest of the stack can reference them.

function_app.tf - *the remediation engine's compute*

A Python Function App on the Consumption (Y1) plan with a System-Assigned Managed Identity - no passwords, no connection strings. Also creates the small Storage Account every Function App needs for its own runtime state (separate from the Week 1 Storage Account being remediated).

rbac.tf - *the least-privilege custom role*

Defines a custom role, 'CloudGuardian Remediator', containing only the exact actions the 6 remediations need - not built-in Contributor - and assigns it to the Function App's and Automation Account's Managed Identities, scoped to the Resource Group only.

event_grid.tf - *the routing layer*

One Event Grid Custom Topic that every finding gets published to, and two subscriptions that split traffic purely on severity: Low/Medium go straight to the Function App; High/Critical go to the Logic App for approval.

logic_app.tf - *the thin orchestration layer*

Deploys a Logic App with a plain HTTP trigger (receives the finding from Event Grid) and ONE HTTP action (forwards it to the Function App's request-approval endpoint). No M365 mailbox, no OAuth connector, no manual authorization step - the Function App does the actual email-sending and decision-handling (see functions/function_app.py and shared/notifications.py). An earlier design tried to put the Approve/Reject branch inside the Logic App itself via the Office 365 Outlook connector, which needs a real Microsoft 365 mailbox and a manual Portal click; this design needs neither.

acs_email.tf - *the email-sending infrastructure*

Provisions Azure Communication Services (Communication Service + Email Service + an Azure-managed sending domain) and grants the Function App's Managed Identity permission to send through it. Fully automated by Terraform - unlike an Office 365 connector, there is no OAuth consent screen to click through.

automation.tf - *the governance layer*

An Azure Automation Account running a PowerShell runbook on a daily schedule, checking whether any of the 6 remediated controls has drifted back to non-compliant.

outputs.tf / terraform.tfvars.example - *what you get back, and the template for your own settings*

outputs.tf prints the Function App name, Event Grid endpoint, Logic App name, and the ACS sender address so you can confirm email is configured correctly. Copy terraform.tfvars.example to terraform.tfvars and fill in YOUR Week 1 resource names - this is the only file you edit directly.

1.2 logic_app/approval_workflow.json

Superseded reference file only - not deployed by the current Terraform. It documents an earlier design that used the Office 365 Outlook connector's 'Send approval email' action (which needed a Microsoft 365 mailbox and a manual OAuth click) and explains what replaced it. The file itself explains the change and points to the files that matter now: terraform/logic_app.tf, terraform/acs_email.tf, functions/function_app.py, and functions/shared/notifications.py.

1.3 automation/Test-RemediationDrift.ps1

A PowerShell runbook that connects using the Automation Account's own Managed Identity (Connect-AzAccount -Identity - no stored credential) and re-checks the live state of all 6 controls. Flags drift; never silently re-fixes it.

1.4 functions/ - the Python remediation engine

host.json / requirements.txt / local.settings.json.example - *Function App runtime configuration*

host.json configures the Functions runtime (extension bundle, 5-minute timeout). requirements.txt pins every Azure SDK package used, including azure-communication-email for the approval/confirmation emails. local.settings.json.example is the template - copy it to local.settings.json before deploying (func azure functionapp publish needs it present to detect this is a Python project); this file is never deployed to Azure and never committed to git.

function_app.py - *the four entry points*

execute_remediation (HTTP, function-key protected) - callable directly for testing/demo. auto_remediate (Event Grid trigger) - Low/Medium findings, no approval. request_approval (HTTP, function-key protected) - called by the Logic App; sends the approval email and returns immediately. approval_decision (HTTP, ANONYMOUS + its own shared-secret check) - reached when a human clicks Approve or Reject in that email; calls the same dispatcher execute_remediation uses, or records a rejection, and returns a small HTML confirmation page either way.

shared/config.py, azure_clients.py, resource_id.py, audit_logger.py, remediation_engine.py, notifications.py - *everything the entry points share*

config.py reads and validates every environment variable (fail-fast, no hardcoded values). azure_clients.py builds authenticated SDK clients via DefaultAzureCredential (Managed Identity in Azure, your `az login` session locally - same code, no branching). resource_id.py parses ARM resource IDs. audit_logger.py wraps every remediation call with structured, queryable start/success/failure logging. remediation_engine.py is the dispatcher - the one place that maps a control_id string to the function that fixes it. notifications.py builds and sends the approval-request and confirmation emails via Azure Communication Services (same Managed Identity, no mailbox, no OAuth), and validates the shared secret the Approve/Reject links carry - see its docstring for exactly why a GET-triggered link needs that secret check instead of just an ordinary Functions key.

remediations/*.py - *the 6 safe fixes, one file each*

storage_public_access.py, storage_encryption.py, diagnostic_logging.py, sql_encryption.py, keyvault_firewall.py, tagging.py. Each has a `remediate(settings, resource\_id, dry\_run, ...)` function, a docstring explaining exactly which CIS/MCSB control it addresses and why it's safe to automate, and dry-run support so you can preview a change before it executes for real.

1.5 privacy_llm/ - the tokenization pipeline

tokenizer.py - *pseudonymize before, restore after*

tokenize_finding() replaces subscription IDs, object IDs, UPNs, resource names, and principal names with deterministic tokens. verify_no_leakage() is a hard guardrail that BLOCKS the outbound LLM call if anything sensitive slipped through. detokenize() restores real values only after the response comes back.

llm_client.py - *the only place allowed to call an LLM*

explain_finding() forces every call through tokenize -> verify -> call -> detokenize, in that order. Two ready-to-use call implementations are included: azure_openai_call (stays inside your Azure tenant boundary end-to-end) and anthropic_call (direct Claude API).

Part 2 - Step by Step: From Zero to a Running Demo

Follow this top to bottom. Total time: 45-60 minutes the first time, most of it Azure provisioning you don't have to watch.

Before You Start

- ☐ Week 1's Terraform stack is deployed and terraform output still works in that folder
- ☐ Azure CLI, Terraform >= 1.7, and Node.js already installed (from Week 1)
- ☐ Python 3.12 installed: `python --version`
- ☐ Azure Functions Core Tools v4 installed: `winget install Microsoft.Azure.FunctionsCoreTools` (Mac: `brew tap azure/functions && brew install azure-functions-core-tools@4`) - close and reopen your terminal after installing so PATH refreshes
- ☐ An inbox you can receive the approval emails in - ANY email address works (Gmail, Outlook.com, a work address). No Microsoft 365 / Exchange Online mailbox is required anywhere in this stack - see the note in `logic_app.tf` and `acs_email.tf` for why.

Step 1 - Gather Your Week 1 Resource Names

From your Week 1 terraform folder, run:

```
terraform output
```

Write down (or copy) the Resource Group name, Log Analytics Workspace name, Key Vault name, Storage Account name, SQL Server name, and SQL Database name. You'll need all six in Step 3.

Step 2 - Unzip / Place the Week 3 Project

```
cd C:\projects\  
copy the week3 folder here, next to (not inside) azure-3tier-terraform\
```

Step 3 - Fill in terraform.tfvars

```
cd week3\terraform  
copy terraform.tfvars.example terraform.tfvars  
notepad terraform.tfvars
```

Fill in the six existing `_*` values from Step 1, your `approver_email` and `notification_email` (your own inbox is fine), and leave everything else at its default.

Step 4 - Initialize and Deploy the Week 3 Stack

```
terraform init  
terraform plan  
terraform apply
```

Type yes when prompted. This takes 5-10 minutes - the Function App, Logic App, Event Grid Topic, and Automation Account are the main wait. When it finishes, run `terraform output` and keep that output visible; you'll need several of these values in the next steps.

Note: On a first-time deploy, this apply can fail specifically on the two `azurerm_eventgrid_event_subscription` resources with a '(401) Unauthorized' webhook validation error - everything else (Function App, ACS, Logic App, RBAC, Automation Account) will still have deployed successfully. This is expected, not a misconfiguration: Event Grid validates a webhook by actually calling it, and your Function App has no code yet at this point in the guide.

Continue to Step 5 to deploy the code, then simply re-run terraform apply - it will pick up only the two subscriptions that failed, and they will validate successfully now that the endpoint responds.

Step 5 - Deploy the Python Function Code

```
cd ..\functions
copy local.settings.json.example local.settings.json
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
func azure functionapp publish <function_app_name from terraform output>
```

Note: The copy step is required, not optional: func azure functionapp publish reads local.settings.json's FUNCTIONS_WORKER_RUNTIME value to detect this is a Python project. Without that file present, publish fails with "Can't determine project language from files." You do not need to fill in real values in it for publish to work (Terraform already set the real app settings in Azure via function_app.tf) - only fill it in later if you want to run func start locally. The first publish can take a couple of minutes while Azure builds the Python dependencies remotely. A successful publish prints all four function names (execute_remediation, auto_remediate, request_approval, approval_decision) with their trigger URLs. If Step 4's terraform apply failed on the Event Grid subscriptions, run terraform apply once more now, from the terraform folder - it will complete cleanly.

Note: If PowerShell blocks .venv\Scripts\activate with a script-execution error, run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass first (scoped to this window only), then retry activate. This has to be re-run in every NEW PowerShell window/tab - it does not persist.

Step 6 - Confirm the Function App Is Live

```
curl -i https://<function-app-name>.azurewebsites.net/api/execute-remediation
-X POST -d "{}"
```

Expect an HTTP 400 with a JSON error about missing fields - NOT a connection timeout or a 404. A 400 with a JSON body means the code deployed and is running; it's rejecting your empty test request correctly.

Step 7 - Verify ACS Email Is Ready (no manual step needed)

Unlike an Office 365 Outlook connector, Azure Communication Services Email needs no OAuth consent click - Terraform provisioned the send permission automatically via the Function App's Managed Identity. Just confirm it looks right:

```
terraform -chdir=week3\terraform output acs_sender_address
```

This should print something like DoNotReply@<a-guid>.azurecomm.net - not a blank value. If it IS blank, `terraform plan` will also have warned about it; see the VERIFY BEFORE APPLY comment in terraform/acs_email.tf for the one-line fix (setting acs_sender_domain_override in terraform.tfvars).

Step 8 - Confirm the Logic App Deployed Correctly

Portal -> your Logic App (logic-cloudguardian-lab-approval) -> Logic App Designer. You should see exactly two steps: the 'When_a_finding_needs_approval' trigger and the 'Request_approval' HTTP action pointing at your Function App's /api/request-approval endpoint. There is nothing to authorize here - if this Logic App shows a red warning icon, it's almost always the Function App URL/key in the action, not a connector auth issue.

Step 9 - Test the Auto-Remediate Path (Low/Medium, no approval)

```
$topicEndpoint = terraform -chdir=week3\terraform output -raw
eventgrid_topic_endpoint
$topicKey = az eventgrid topic key list --name <topic-name> --resource-group
<rg> --query key1 -o tsv

curl -X POST "$topicEndpoint/api/events" `
  -H "aeg-sas-key: $topicKey" -H "Content-Type: application/json" `
  -d
'[{ "id": "t1", "eventType": "CloudGuardian.Finding.Detected", "subject": "test", "eventTime": "2026-07-11T00:00:00Z", "dataVersion": "1.0", "data": { "finding_id": "t1", "control_id": "tagging", "remediation_type": "tagging", "severity": "Low", "resource_id": "/subscriptions/<sub>/resourceGroups/<rg>" } } ]'
```

Within a few seconds, check the Resource Group's tags in the Portal - the 4 required tags should appear. This is the safest control to test first since tagging can't break anything.

Step 10 - Test the Approval Path (High/Critical)

Repeat Step 9's curl command with severity set to "High" and control_id/remediation_type set to "sql_encryption". Check the inbox you set as approver_email for an email from your ACS sender address, then click the Approve link (a plain hyperlink, not an Outlook Actionable Message button - see the design note in functions/shared/notifications.py for why). You should land on a small confirmation page, and the SQL Database's Transparent Data Encryption blade in the Portal should flip to Enabled. A second email confirms the outcome.

Note: Any inbox works here - Gmail, Outlook.com, a work address, anything. ACS Email only needed a special mailbox for *SENDING*; there was never a restriction on which address *RECEIVES* the approval email.

Step 11 - Check the Audit Trail

Portal -> your Log Analytics Workspace -> Logs -> paste and run:

```
traces
| where customDimensions.event_type == "remediation_result"
| project timestamp, tostring(customDimensions.control_id),
tostring(customDimensions.outcome), tostring(customDimensions.resource_id)
| order by timestamp desc
```

Note: If this returns nothing, give it 2-3 minutes - Application Insights ingestion has a short delay - then re-run.

Step 12 - Test the Governance Runbook

Portal -> your Automation Account -> Runbooks -> Test-RemediationDrift -> Start (fill in the same parameters Terraform's automation.tf already wired up automatically for the SCHEDULED run - for a manual test run, paste in your Week 1 resource names). Review the job output for a 'No drift detected' or a listed drift finding.

Note: Optional next step, not required for the demo: create an Azure Monitor Alert Rule on the Automation Account's JobLogs category (Warning severity) to get an email/Teams notification automatically instead of checking the job output by hand.

Step 13 - Try the Privacy-Preserving LLM Pipeline Locally

```
cd ..\privacy_llm
pip install -r requirements.txt
```

```
python -c "from tokenizer import tokenize_finding, verify_no_leakage,
detokenize;
f={'finding_id':'f1','control_id':'storage_public_access','upn':'you@example.
com','resource_id':'/subscriptions/x/resourceGroups/rg/providers/
Microsoft.Storage/storageAccounts/stdemo'}; s,t=tokenize_finding(f);
print(s)"
```

You should see the subscription/resource/UPN fields replaced with tokens like RESOURCE_ID_1, UPN_1. Wire llm_client.py's azure_openai_call() or anthropic_call() to your own Week 2 LLM credentials to run the full explain_finding() round trip.

Step 14 - Shut Down When You're Done for the Day

The Function App (Consumption) and Logic App (Consumption) scale to zero and cost nothing idle. The Automation Account's daily schedule and the small Function/Logic App storage accounts have negligible cost. If you want to fully tear down Week 3 without touching Week 1:

```
cd week3\terraform
terraform destroy
```

Note: This does NOT touch your Week 1 resources - Week 3's Terraform state only contains the resources this stack created (Function App, Event Grid, Logic App, Automation Account, RBAC assignments). Re-running terraform apply rebuilds Week 3 in a few minutes against your still-running Week 1 environment.

Troubleshooting Common Errors

Error message contains...	What it means	Fix
request_approval returns 500 / no email arrives	ACS Email permission hasn't propagated yet, or acs_sender_address is blank	Wait ~5 minutes after first apply for RBAC propagation; re-check terraform output acs_sender_address (Step 7); check Application Insights traces for the exact EmailClient error
approval_decision returns Link not valid	The secret query parameter doesn't match CALLBACK_SHARED_SECRET, or you're testing with a hand-typed URL	Only click the actual link from the email - don't retype it; if you redeployed Terraform, random_password.callback_secret may have rotated, invalidating old emails' links
curl to the Function returns a connection timeout	Function App is still cold-starting, or the code didn't deploy	Wait 30 seconds and retry; re-run func azure functionapp publish and watch for errors
Event Grid subscription shows validation pending	The webhook endpoint didn't answer Event Grid's handshake in time	Re-run terraform apply - Terraform re-triggers the subscription creation
terraform apply fails creating azure_rm_eventgrid_event_subscription with (401) Unauthorized	Expected on a first-time deploy: Event Grid validates a webhook by calling it, but the Function App has no code yet at Step 4	Continue to Step 5 and deploy the function code, then re-run terraform apply - it will pick up only the failed subscriptions and they will validate successfully now
terraform apply fails on azure_rm_communication_service / azure_rm_email_communication_service with MissingSubscriptionRegistration	Your subscription has never used the Microsoft.Communication resource provider before	az provider register --namespace Microsoft.Communication, then poll az provider show --namespace Microsoft.Communication --query registrationState -o tsv until it says Registered, then retry
terraform apply fails creating azure_rm_automation_account with Free Trial and Student subscriptions cannot create accounts in this location	Free Trial/Student Azure subscriptions restrict Automation Accounts to a fixed region allow-list independent of where the rest of the stack lives	Set automation_account_location in terraform.tfvars to one of: eastus, eastus2, westus, northeurope, southeastasia, japanwest (see variables.tf)
Incorrect attribute value type: map of string required, but have string (headers on azure_rm_logic_app_action_http)	jsonencode() was used on the headers argument, which takes a native HCL map(string) directly - unlike body, which does need jsonencode()	In logic_app.tf, change headers = jsonencode({ ... }) to a plain headers = { ... } map
Not enough list items (site_config.0.cors.0.allowed_origins requires 1 item minimum)	An empty cors { allowed_origins = [] } block isn't valid HCL for no CORS - Azure requires at least one entry if the block exists at all	Delete the whole cors { } block from function_app.tf's site_config - none of this stack's callers need it (see the comment left in its place)
func : term not recognized (right after installing Azure Functions Core Tools)	The current terminal window still has the old PATH from before the install	Close this window entirely and open a brand-new one, then retry func --version
.venv\Scripts\activate cannot be loaded because running scripts is disabled	PowerShell's default execution policy blocks unsigned local scripts, including venv's own activation script	Run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, then retry activate - this is scoped to the current window and must be repeated in each new one
Can't determine project language from files (func azure functionapp publish)	local.settings.json.example was never copied to local.settings.json - func reads FUNCTIONS_WORKER_RUNTIME from that file to detect this is a Python project	Run copy local.settings.json.example local.settings.json in the functions folder, then retry publish
transparentDataEncryptions.get()	Wrong database name, or TDE	Double-check

Error message contains...	What it means	Fix
raises a 404	current resource name changed in a newer API version	existing_sql_database_name in terraform.tfvars matches exactly; confirm with az sql db show
Automation runbook fails with Connect-AzAccount : Run Connect-AzAccount to login	The runbook wasn't given -Identity, or the Automation Account's Managed Identity role assignment hasn't propagated yet	Confirm rbac.tf's automation_remediator role assignment applied cleanly; RBAC propagation can take up to 5 minutes after first apply
PrivacyLeakError raised on every call	A real value is present in a text field that isn't a recognized structured field	Add the field to SENSITIVE_FIELDS in tokenizer.py, or pre-tokenize it before calling explain_finding()
terraform plan errors on mail_from_sender_domain (Unsupported attribute)	The azurearm provider version you have renamed/changed this computed attribute	Check the current azurearm_email_communication_service_domain docs on the Terraform Registry for the right attribute name, or set acs_sender_domain_override in terraform.tfvars using az communication email domain show instead - see the comment in acs_email.tf

Command Cheat Sheet

Command	What it does
terraform apply	deploy/update the Week 3 stack
terraform output	see Function App name, Event Grid endpoint, Logic App name
func azure functionapp publish <name>	deploy the Python remediation code
az eventgrid topic key list --name <t> -g <rg>	get the key needed to publish test events
curl -X POST "\$endpoint/api/events" ...	publish a test finding to trigger the pipeline
az automation runbook start ...	manually trigger the drift-check runbook
terraform destroy	tear down Week 3 only (Week 1 untouched)